

# DEEP LEARNING

## Deep Neural Networks Part III

Subhankar Roy

Department of Management, Information and Production Engineering (DIGIP)  
University of Bergamo

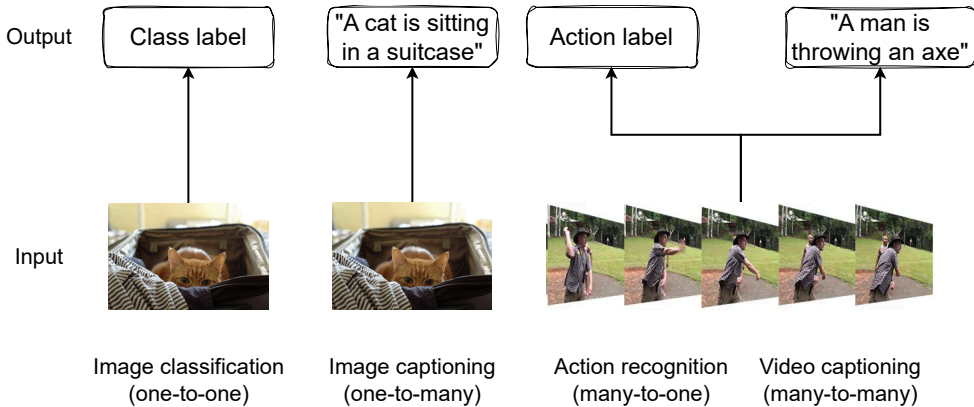
# From Grids to Sequences

---

- Convolutional Neural Networks (CNNs) are:
  - ▶ Designed for spatial data (e.g., images).
  - ▶ Good at capturing local patterns with shared filters.
  - ▶ However, not designed for handling sequences or time-dependent data.
- We need something new because:
  - ▶ Many tasks involve **sequential data** (e.g., text, speech, time series, video).
  - ▶ We need models that can remember **past inputs** and understand **context**.
- Lets compare the two sentences with the same word "bank" in them:
  - ▶ "I went to the **bank** to withdraw some **money**."
  - ▶ "I sat on the **bank** and watched the **water** flow by."
- To understand the correct meaning of the word "**bank**", the model must understand (or be aware of) the surrounding words highlighted in yellow, i.e., the **context**.

# Processing Sequences

- Sequences can arise at the input, the output or both.



# Table of Contents

---

## 1. Recurrent Neural Networks

## 2. Sequence to Sequence Modelling

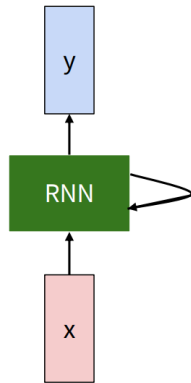
- 2.1 Seq2Seq with RNN
- 2.2 Seq2Seq with RNN and Attention
- 2.3 Attention
- 2.4 Encoder-Decoder Transformer

# Recurrent Neural Networks

# Recurrent Neural Networks

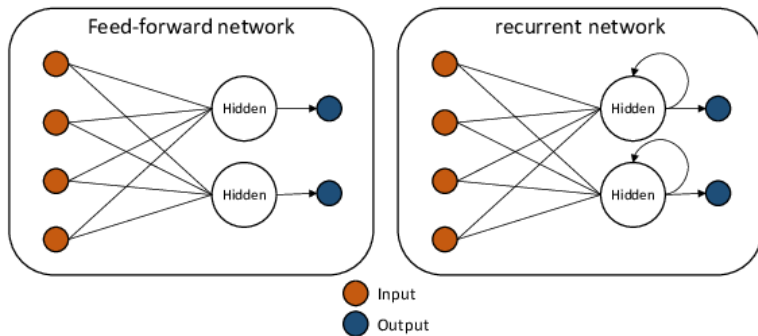
---

- **Recurrent Neural Network (RNN)** is a type of neural network that can handle sequences. It has the following components:
  - ▶ Maintains a **internal state** to “remember” previous inputs → introduces the notion of time.
  - ▶ Think of the **internal state** (shown by the arrow in the middle) as the a “*compressed summary*” of what it has seen in the previous time steps.
  - ▶ Outputs at each time step depend on **current input** + **past context**.
- **Key intuition:** While reading a sentence you understand each word based on the words before it. RNNs try to do the same.



# Feed Forward Versus Recurrent Neural Networks

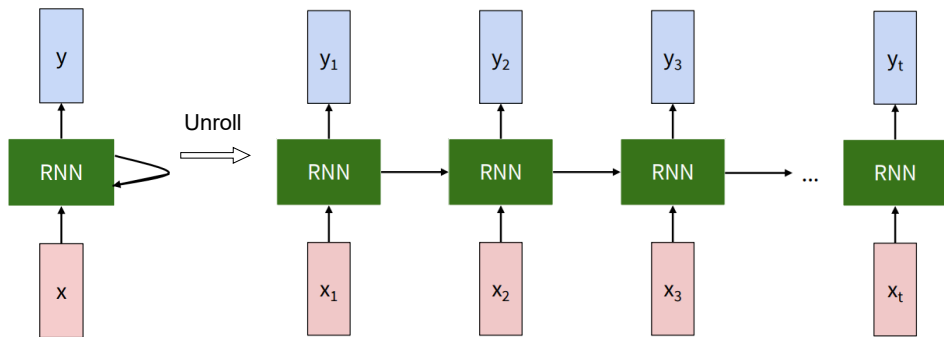
---



- In a FFN a hidden layer always receives input from the previous layer. That is why we call it “feed-forward”.
- In a RNN the output also depends on the internal state of the model, ie, has self-loops. That is why its called “recurrent”.

# Unrolling RNNs

---





# RNN Hidden State Update

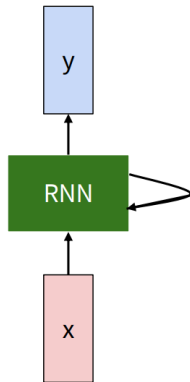
---

- The internal state of the RNN gets updated at every timestep  $t$  as follows:

$$h_t = f_{\theta}(h_{t-1}, x_t),$$

where:

- ▶  $x_t$  is the input at the current time step (a typical input as in MLP/CNN).
- ▶  $h_{t-1}$  is the old (internal) state.
- ▶  $f_{\theta}$  is a neural network, parameterized by parameters  $\theta$  (not any different than a standard MLP/CNN).
- ▶  $h_t$  is the new hidden state.



# RNN Output Generation

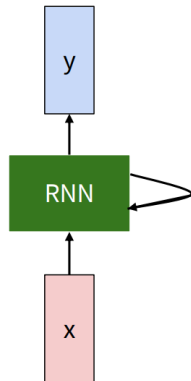
---

- The model outputs a vector  $y$  at every time step  $t$  as follows:

$$y_t = f_{\phi}(h_t),$$

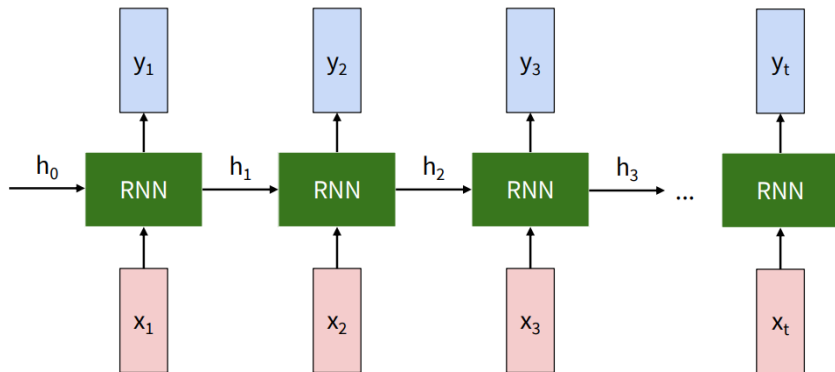
where:

- ▶  $h_t$  is current hidden state.
  - ▶  $f_{\phi}$  is another neural network, parameterized by parameters  $\phi$ .
  - ▶  $y_t$  is the current output.
- In practice the RNN is just a single neural network, with dedicated parameters for computing the hidden state and the output.



# Unrolling RNN and its States

---



- The same function and the same set of parameters are used at every time step.

# RNN Internals

---

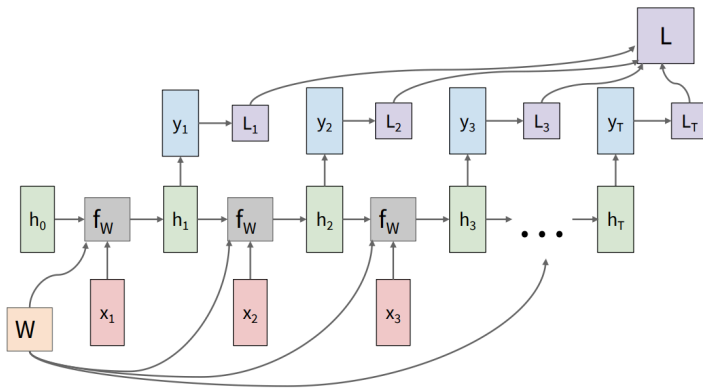
- More concretely, a simple RNN is described by the following two equations:

$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t),$$
$$y_t = W_{hy}h_t,$$

where:

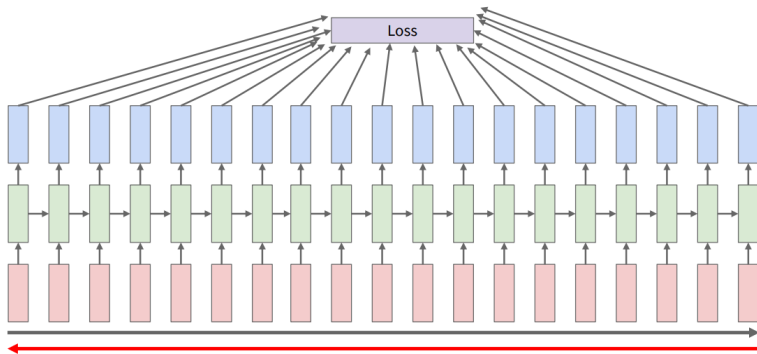
- ▶  $\tanh$  is the non-linear activation function.
  - ▶  $W_{hh}$  is a linear layer that transforms the features of the past hidden state  $h_{t-1}$ .
  - ▶  $W_{xh}$  is a linear layer that transforms the features of the input  $x_t$ .
  - ▶  $W_{hy}$  is a linear layer that transforms the features of the current hidden state  $h_t$ .
- These transformations are no different than the ones we find in MLP or CNN.

# RNN Forward Pass



- Note that the same  $W$  is used at every timestep.
- We compute a loss  $L_t$  at each time step, and we sum them to get the final loss.

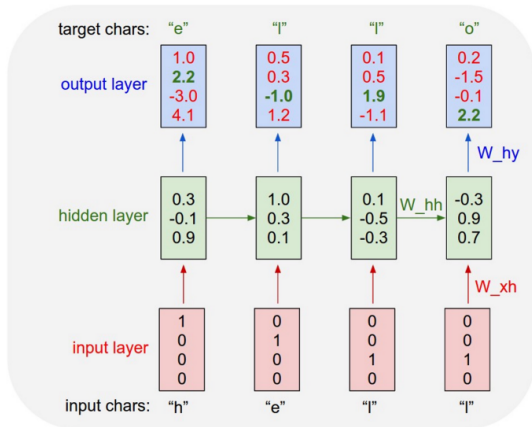
# RNN Backpropagation Through Time



- Forward the entire sequence to compute loss, then backpropagate through entire sequence to compute gradients.

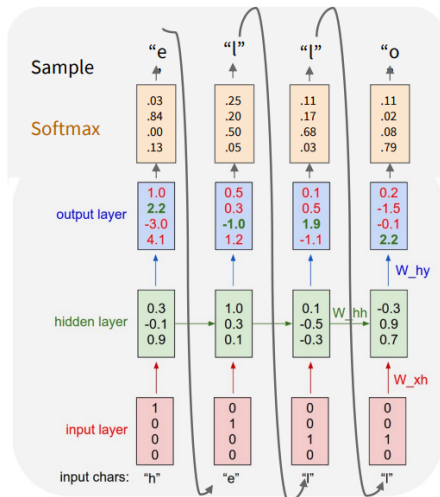
# A Simple Language Model using RNN

- Task: to predict the next character in a sequence.
- We have a training sequence: "hello".
- Vocabulary: ['h', 'e', 'l', 'o']. We use **one-hot encoding** to represent a character.



# A Simple Language Model using RNN

- At test-time sample characters one at a time, feed it back to the model as input.





# RNN: Example Generations

PANDARUS:

Alas, I think he shall be come approached and the day  
When little strain would be attain'd into being never fed,  
And who is but a chain and subjects of his death,  
I should not sleep.

Second Senator:

They are away this miseries, produced upon my soul,  
Breaking and strongly should be buried, when I perish  
The earth and thoughts of many states.

DUKE VINCENTIO:

Well, your wit is in the care of side and that.

Second Lord:

They would be ruled after this chamber, and  
my fair nudes begun out of the fact, to be conveyed,  
Whose noble souls I'll have the heart of the wars.

Clown:

Come, sir, I will make did behold your worship.

VIOLA:

I'll drink it.

*Proof.* Omitted. □

**Lemma 0.1.** *Let  $\mathcal{C}$  be a set of the construction.*

*Let  $\mathcal{C}$  be a gerber covering. Let  $\mathcal{F}$  be a quasi-coherent sheaves of  $\mathcal{O}$ -modules. We have to show that*

$$\mathcal{O}_{\mathcal{O}_X} = \mathcal{O}_X(\mathcal{L})$$

*Proof.* This is an algebraic space with the composition of sheaves  $\mathcal{F}$  on  $X_{\text{étale}}$  we have

$$\mathcal{O}_X(\mathcal{F}) = \{\text{morph}_1 \times_{\mathcal{O}_X} (\mathcal{G}, \mathcal{F})\}$$

where  $\mathcal{G}$  defines an isomorphism  $\mathcal{F} \rightarrow \mathcal{F}$  of  $\mathcal{O}$ -modules. □

**Lemma 0.2.** *This is an integer  $Z$  is injective.*

*Proof.* See Spaces, Lemma ?? □

**Lemma 0.3.** *Let  $S$  be a scheme. Let  $X$  be a scheme and  $X$  is an affine open covering. Let  $\mathcal{U} \subset X$  be a canonical and locally of finite type. Let  $X$  be a scheme. Let  $X$  be a scheme which is equal to the formal complex.*

*The following to the construction of the lemma follows.*

*Let  $X$  be a scheme. Let  $X$  be a scheme covering. Let*

$$b: X \rightarrow Y' \rightarrow Y \rightarrow Y' \times_X Y \rightarrow X.$$

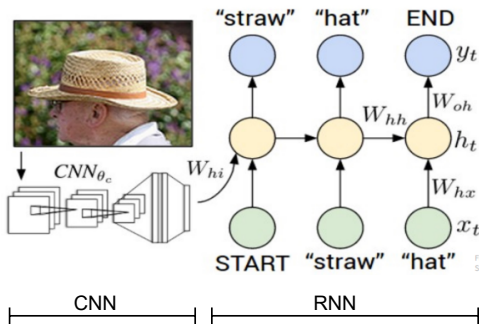
*be a morphism of algebraic spaces over  $S$  and  $Y$ .*

*Proof.* Let  $X$  be a nonzero scheme of  $X$ . Let  $X$  be an algebraic space. Let  $\mathcal{F}$  be a quasi-coherent sheaf of  $\mathcal{O}_X$ -modules. The following are equivalent

- (1)  $\mathcal{F}$  is an algebraic space over  $S$ .
- (2) If  $X$  is an affine open covering.

Consider a common structure on  $X$  and  $X$  the functor  $\mathcal{O}_X(\mathcal{U})$  which is locally of finite type. □

# RNN: Captioning Images



- We can combine a CNN and a RNN to create a hybrid model for the task of image captioning, ie, predict a caption (or sentence) given an input image.
- The features from the CNN acts as the initial hidden state of the RNN.
- There are two special inputs/outputs:  $START$  and  $END$ , which indicate the start and end of a sentence, respectively.

# Variants of RNN

---

- RNN suffers from several problems such as:
  - ▶ vanishing gradients.
  - ▶ when the sequence length increases, it fails to capture information that is further back in time.
- There are more advanced variants of RNNs, but share its working principle:
  - ▶ Long Short Term Memory (LSTM)
  - ▶ Gated Recurrent Unit (GRU)
  - ▶ Extended Long Short-Term Memory (xLSTM)
  - ▶ Many more...

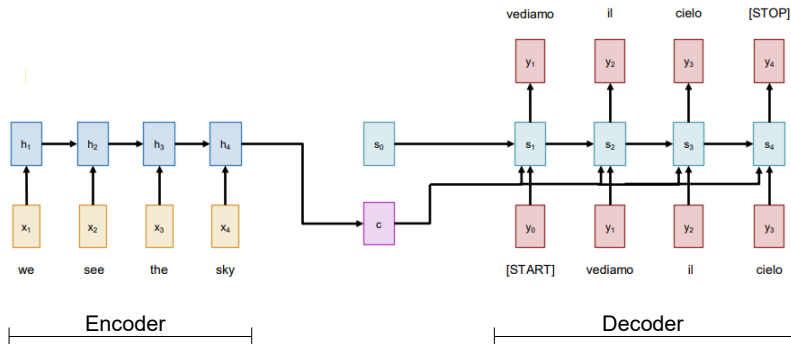
## Sequence to Sequence Models

# Sequence to Sequence (Seq2Seq) Modelling

---

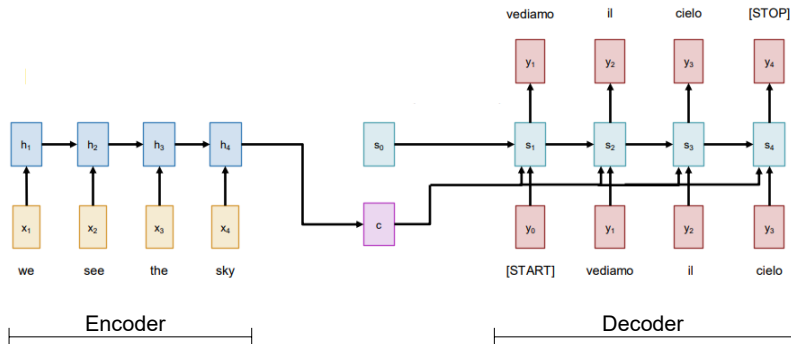
- **Seq2Seq** is a neural network architecture that takes a sequence as input and produces another sequence as output.
- Example:
  - ▶ Input sequence: *"I am a student."*
  - ▶ Output sequence: *"Je suis étudiant."*
- Why do we need sequence to sequence modelling?
  - ▶ **Machine Translation**: English → Italian.
  - ▶ **Summarization**: Long article → short summary.
  - ▶ **Chatbots**: Input query → appropriate response.
- Typically consists of two parts:
  - ▶ **Encoder**: processes (or encodes) the input sequence.
  - ▶ **Decoder**: generates the output sequence.

# Seq2Seq with RNN



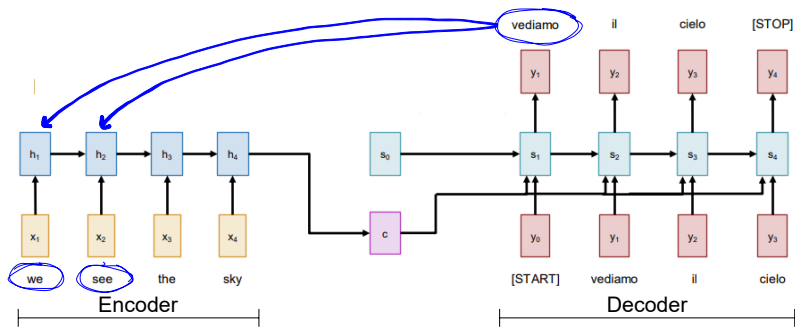
- Consider the **machine translation** problem of translating from English to Italian.
- The Encoder encodes the English sentence and outputs  $h_4$ . The **context vector**  $c$  contains the summarized representation of the English sentence (often  $c = h_T$ ).
- At timestep  $t$ , the Decoder takes as input: the previous **decoder state**  $s_{t-1}$ , the **context vector**  $c$ , and the **current input**  $y_t$ ; and outputs the next word  $y_{t+1}$ .

# Seq2Seq with RNN



- The input sequence is compressed into a single context vector  $c$ .
- **Drawback:** When the input sequence length is very long, the context vector may not adequately contain information from earlier in the sequence.
- **Solution:** Look back at the whole input sequence at each step of the output.

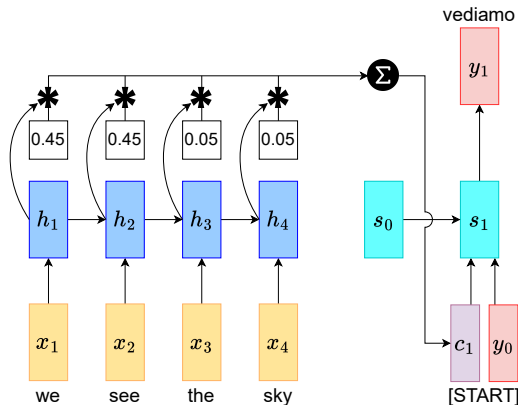
# Seq2Seq with RNN and Attention



- In the above example, to decode the word “vediamo” in the output, we do *not* need to look at the whole input sequence.
- To decode “vediamo”, the model should only pay **attention** to the words “we see” more than any other words in the input sequence.
- This is the main idea behind what we call the **attention mechanism**. It is the one of the main ingredients in all major AI models.



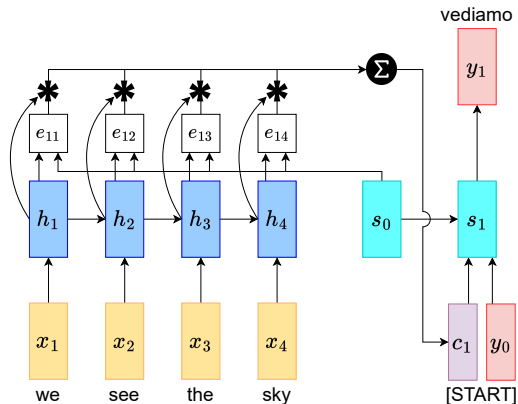
# Seq2Seq with RNN and Attention



- Compute a separate context vector at each timestep (e.g.,  $c_1$  for decoding  $y_1$ ).
- $c_1$  is computed by taking a **weighted sum** of the hidden states  $h_t$ .
- For "vediamo", assign higher weights to  $h_1$  and  $h_2$ , and lower weights for  $h_3$  and  $h_4$ .

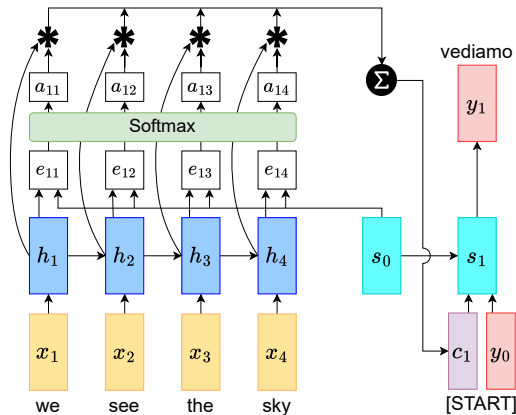
# Seq2Seq with RNN and Attention

- Q: How do we get the “weights”? A: [Learn them!](#)



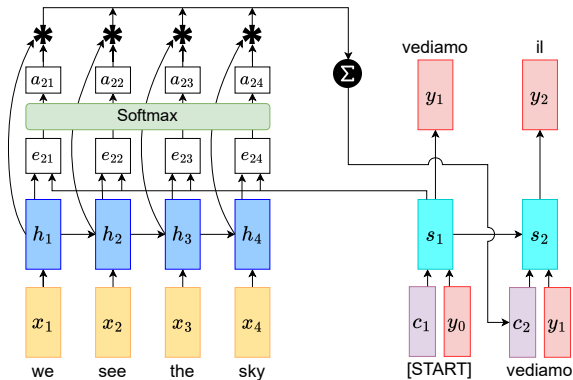
- The [alignment scores](#)  $e$ , which are scalar values, tell us how similar is a decoder hidden state  $s$  to the encoder hidden state  $h$ .
- The alignment is learned by the network automatically.

# Seq2Seq with RNN and Attention



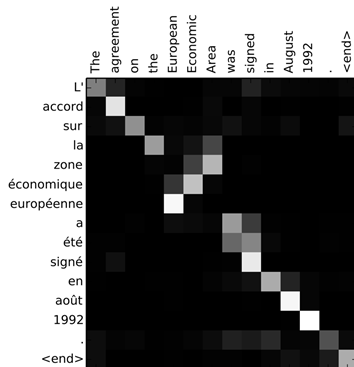
- In practice, we normalize alignment scores  $e$  to get **attention weights**  $a$ .
- Softmax makes sure that the attention weights:  $0 \leq a \leq 1, \sum_i a_i = 1$ .

# Seq2Seq with RNN and Attention



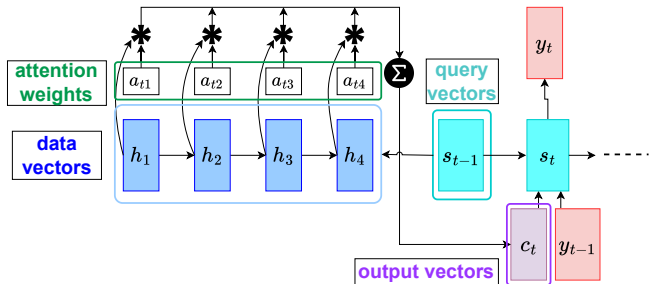
- Use  $s_1$  to compute a new context vector  $c_2$ . This means, compute new alignment scores and new attention weights.
- To decode "il" the model must assign high attention weights  $a_{23}$ , which corresponds to the word "the".

# Visualizing Attention Weights



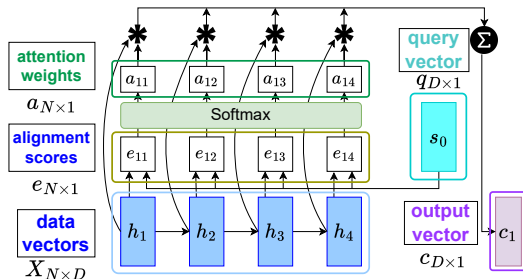
- **Input** (en): "The agreement on the European Economic Area was signed in August 1992."
- **Output** (fr): "L'accord sur la zone économique européenne a été signé en août 1992."
- Notice the attention weights are brighter when translating the relevant word(s).

# Attention



- At a high-level we are dealing with a generic operation called **attention**:
  - ▶ **Query vectors** (decoder RNN states) and **data vectors** (encoder RNN states) interact to get **attention weights**.
  - ▶ **Data vectors** are weighted by **attention weights** and get transformed to **output vectors** (Context states).
- Each **query** attends to all **data vectors** and gives one **output vector**.

# Generalizing Attention



- Let  $N$  denote the length of the input sequence and  $D$  the feature dimension.
- Similarities (or alignment scores):  $e_i = q \cdot X_i$ . This is also called **dot product similarity**. Note, by abusing notations, we are denoting  $h_i$  by  $X_i$ .
- Attention weights:  $\mathbf{a} = \text{softmax}(\mathbf{e})$ .
- Output vector:  $\mathbf{y} = \sum_{i=1}^N a_i X_i$ . We will denote  $c$  as  $\mathbf{y}$ .

# Generalizing Attention

---

- We can have multiple **query vectors**:  $Q_{M \times D}$ , where  $M$  is the number of query vectors.
- Recall, the input matrix is  $X_{N \times D}$ .
- Similarities:  $E_{M \times N} = QX^T$ .
- Attention weights:  $A_{M \times N} = \text{softmax}(E, \text{dim} = 1)$ .
- Output vectors:  $Y_{M \times D} = AX$ . For  $M$  **query vectors** we get  $M$  **output vectors** back.
- Q: What are we exactly learning here? Input  $X$ , dot product similarity, and softmax are not “learnable”.
- A: We introduce **learnable parameters**.



# Cross-Attention

- We introduce learnable parameters:

- ▶ **key matrix**  $W_K$ ,
- ▶ **value matrix**  $W_V$

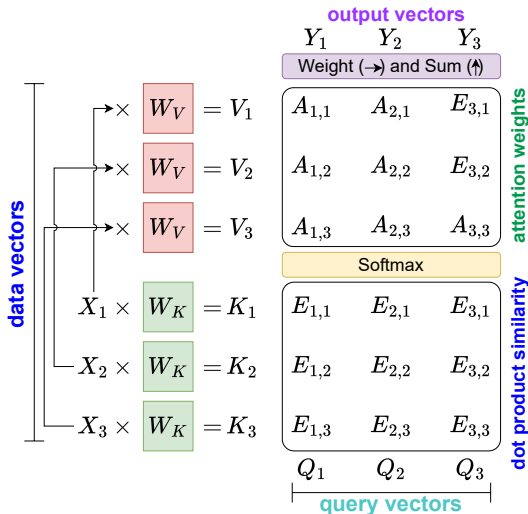
- Computations are:

- ▶ Keys:  $K = XW_K$
- ▶ Values:  $V = XW_V$
- ▶ Similarities:  $E = QK^\top$
- ▶ Attention weights:  
 $A = \text{softmax}(E)$
- ▶ Outputs:  $Y = AV$

- More compactly we can write:

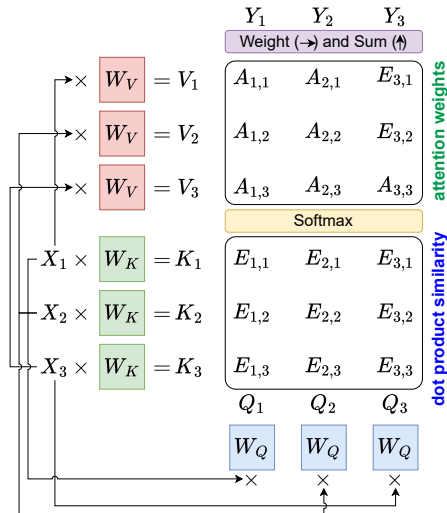
$$Y = \text{softmax}(QK^\top)V$$

- Each **query** produces one **output**, which is a mix of information in the **data vectors**.



# Self-Attention

- We can use the idea and apply it within a given sequence, instead of across two sequences.
- We introduce another learnable matrix: **query matrix**  $W_Q$ .
- Computations remain unchanged except:
  - Queries:  $Q = XW_Q$
- Each **input** produces one **output**, which is a mix of information from all **inputs**.
- Intuitively, **self-attention** lets each word in a sentence look at and learn from other words.



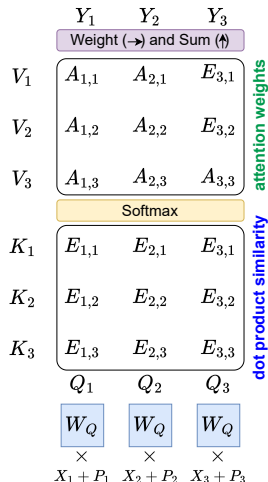
# Self-Attention

---

- Let's understand what self-attention is trying to achieve with the following example:
- *"A robot must obey the orders given to **it** by human beings except where **such orders** would conflict with the **First Law**."*
- When a model processes this sentence, it has to know that:
  - ▶ **it** refers to the robot
  - ▶ **such orders** refers to the earlier part of the law, namely "the orders given it by human beings"
  - ▶ The **First Law** refers to the entire First Law
- Self-attention helps the model understand context by identifying which words (e.g., "robot") are most relevant to the word (e.g., "it") being processed.
- It assigns scores (i.e., attention weights) to each word's relevance and combines their vector representations to capture meaning.

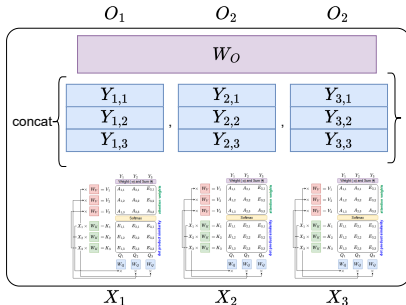
# Self-Attention

- An issue with self-attention is that it is **permutation equivariant**.
- Meaning, if the order of the input words is permuted, then the output remains the same, but permuted.
- We add **positional encoding**  $P_i$  to each input  $X_i$ .
- $P_i$  is a vector that is a fixed function of the position of the word in the sentence. Now each input  $(X_i + P_i)$  will be dependent on its position.



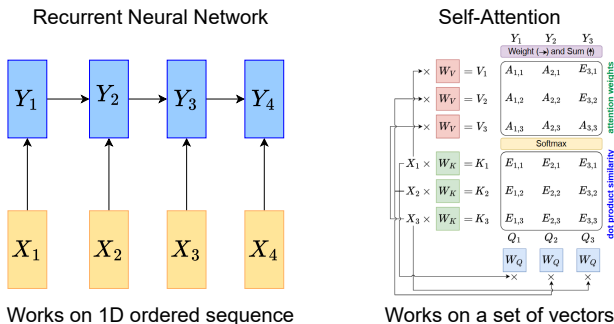
# Multi-head Self-Attention

- Run  $H$  copies of independent self-attention layers (or **heads**) in parallel  $\rightarrow$  multi-headed. Using an example of  $H = 3$ , each head receives  $X_1, X_2, X_3$ .



- Each head has its independent weights. Intuitively, each head captures different information in different positions of the sequence.
- Stack the outputs from each head and multiply with the **output matrix**  $W_O$ .
- Output:  $O = YW_O$ .

# Processing Sequences: RNN vs Self-Attention (SA)



- RNNs are inherently **sequential** because they process one time step at a time, and each step depends on the previous one → no parallelization across time steps.
- On the contrary, SA sees **all input vectors at once**, and learn dependencies regardless of distance, using attention weights → parallelizeable across time steps.
- This is a significant advantage: GPUs are specialized for parallel compute.

# Transformer

- Introduced by Vaswani et al., 2017 in “Attention is All You Need”.
- Transformer is based entirely on **self-attention** and **cross-attention**, not recurrence.
- Processes entire sequences in parallel, enabling fast and scalable training.
- Transformers have transformed AI. Powering chatbots, translators, code generators, and more.

---

## Attention Is All You Need

---

Ashish Vaswani\*  
Google Brain  
avaswani@google.com

Noam Shazeer\*  
Google Brain  
noam@google.com

Niki Parmar\*  
Google Research  
nikip@google.com

Jakob Uszkoreit\*  
Google Research  
usz@google.com

Llion Jones\*  
Google Research  
llion@google.com

Aidan N. Gomez\*<sup>1</sup>  
University of Toronto  
aidan@cs.toronto.edu

Lukasz Kaiser\*  
Google Brain  
lukaszkaizer@google.com

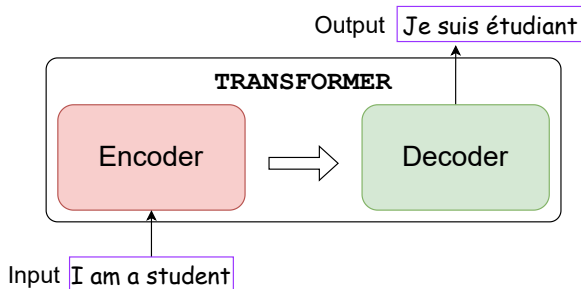
Illia Polosukhin\*<sup>1</sup>  
illia.polosukhin@gmail.com

### Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including ensembles, by over 2 BLEU. On the WMT 2014 English-to-French translation task, our model establishes a new single-model state-of-the-art BLEU score of 41.8 after training for 3.5 days on eight GPUs, a small fraction of the training costs of the best models from the literature. We show that the Transformer generalizes well to other tasks by applying it successfully to English constituency parsing both with large and limited training data.

# Transformer

---



- Transformer has an **encoder-decoder** architecture (similar to Seq2Seq modelling with RNN), but uses self-attention and cross-attention mechanism.
- In the original paper, it was designed for machine translation (e.g., en  $\rightarrow$  fr).
- Later, other variants became popular:
  - ▶ Encoder-only Transformer (eg, BERT) for text classification
  - ▶ Decoder-only Transformer (eg, ChatGPT) for text generation



# Applications of Transformers

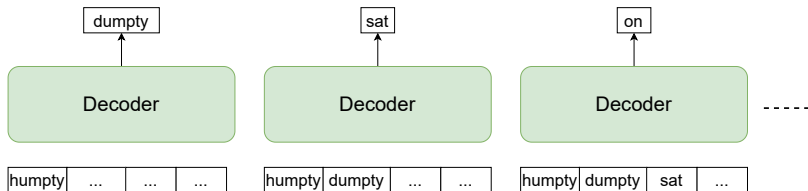
---

- Transformers have become the de facto architecture in many applications:
  - ▶ Natural Language Processing
  - ▶ Computer Vision
  - ▶ Audio Processing
  - ▶ Multimodal Learning
- Transformers are widely adopted in various applications because they are:
  - ▶ Scalable
  - ▶ Computations can be parallelized
  - ▶ Models long range dependencies
  - ▶ Makes minimal assumptions about the input data
  - ▶ Unified architecture across different input modalities

# Transformer For Language Modelling

---

- Very big Transformers trained on a large corpus of text are often called **Large Language Models** (LLMs).
- State-of-the-art LLMs (e.g., ChatGPT) mostly adopt **decoder-only** architecture.
- At inference time, one token is produced at a time. The output of the model is then appended to the previous input to produce the next token.
- This goes on until the [EOS] token is produced.

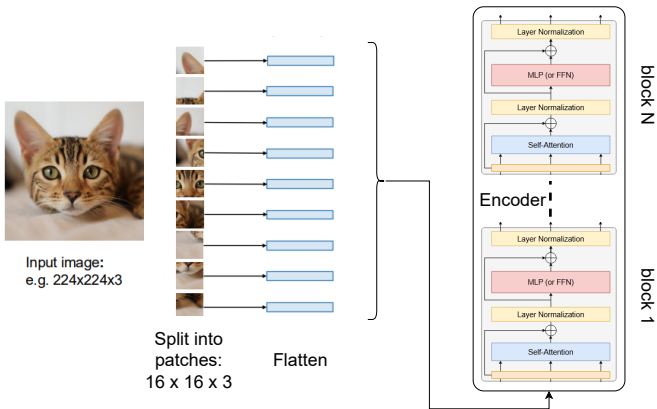


---

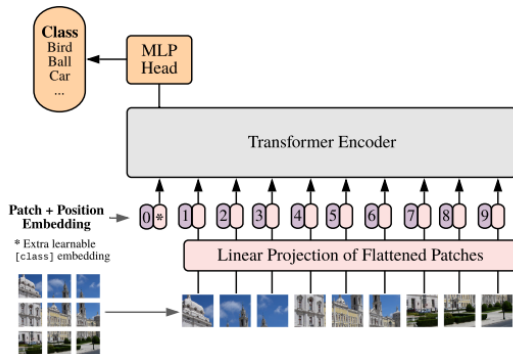
<sup>0</sup>Pre-trained LLM: <https://app.hyperbolic.xyz/models/llama31-405b-base-bf-16>

# Vision Transformer (ViT)

- Treats **image patches as tokens**, just like words in natural language.
- Patches are flattened and embedded into vectors.
- Passed through standard Transformer encoder blocks.



# Vision Transformer (ViT)



- Adds a special **learnable token** called [CLS], which stands for “classification” token.
- The final embedding of [CLS] is treated as the summary representation of the entire sequence (because all the tokens interact through self-attention). The [CLS] vector is passed to a classifier head (e.g., a MLP + softmax) to make predictions.

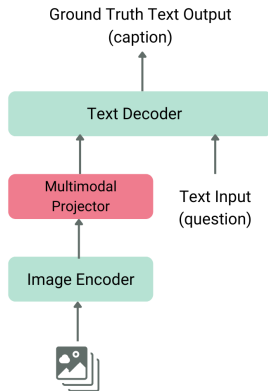
# Multimodal Learning

---

- Vision Language Models combine visual understanding with natural language generation.
- Can see (images) and talk (generate text), enabling tasks like:
  - ▶ Image captioning
  - ▶ Visual question answering (VQA)
- Image is processed by a ViT to extract image features.
- A decoder-only LLM generates text conditioned on image features + text input.<sup>a</sup>

<sup>a</sup>Demo:

<https://huggingface.co/spaces/badayvedat/LLaVA>



# References and Links

---

- For additional reading materials refer to the following links
  - ▶ [“The Illustrated Transformer”](#)<sup>1</sup>.
  - ▶ [“Attention? Attention!”](#)<sup>2</sup>.
  - ▶ [“The Illustrated GPT-2”](#)<sup>3</sup>.
  - ▶ [“Everything About Transformers”](#)<sup>4</sup>.
- Some slides have been adapted from J. Johnson.

---

<sup>1</sup><https://jalammar.github.io/illustrated-transformer/>

<sup>2</sup><https://lilianweng.github.io/posts/2018-06-24-attention/>

<sup>3</sup><https://jalammar.github.io/illustrated-gpt2/>

<sup>4</sup><https://www.krupadave.com/articles/everything-about-transformers>